**UNOFFICIAL — COMMUNITY-DERIVED SPECIFICATION**

# Panavox PSS-12 INV

## Serial Communication Protocol Specification

UART Serial Control Interface (W950 Connector)

*This is not an official Panavox document. This specification was derived from open-source code and empirical testing. See Disclaimer section for full details.*

**Model:** Panavox PSS-12 INV (Split Inverter AC)

**Interface:** UART (3.3V logic via W950 WiFi connector), 9600 baud, 8N1

**Controller:** ESP32-S3

**Version:** 1.0

**Date:** March 31, 2026

**Author:** Farid Elias

# Table of Contents

**1. Overview**

**2. Physical Layer**

**3. Frame Structure**

- 3.1 Start and End Delimiters
- 3.2 Byte Stuffing
- 3.3 Checksum Calculation
- 3.4 Command Frame Layout (Controller to AC)
- 3.5 Response Frame Layout (AC to Controller)
- 3.6 Master/Slave Communication Model
- 3.7 Command-Response Behavior
- 3.8 Response Frame Transmission Timing
- 3.9 Query Queuing During Active Response

**4. Command Reference**

- 4.1 Power Control
- 4.2 Operating Modes
- 4.3 Fan Speed
- 4.4 Temperature Setting
- 4.5 Swing / Vane Control
- 4.6 Presets (Turbo / Energy Save)
- 4.7 Display Control
- 4.8 Sleep Modes
- 4.9 Temperature Unit Switching
- 4.10 Status Request

**5. Status Response Parsing**

- 5.1 Device_Status Structure
- 5.2 Mode Status Values
- 5.3 Fan Speed Status Values
- 5.4 Temperature Decoding
- 5.5 Swing Status
- 5.6 Sensor Data Fields
- 5.7 Feature Flags
- 5.8 Fahrenheit/Compensation Byte

**6. State Machine and Timing**

**7. Command Payload Byte Map**

**8. Appendix: Full Frame Listing**

**Disclaimer**

**Document History**

# 1. Overview

This document specifies the serial communication protocol used to control and monitor the Panavox PSS-12 INV split inverter air conditioning unit. The protocol operates over a direct serial UART connection between an ESP32-S3 microcontroller and the indoor unit's main control board, via the W950 WiFi module connector. A simple 2N2222 transistor-based level shifter bridges the voltage domains between the ESP32 and the AC board.

The protocol supports bidirectional communication: the controller sends command frames to change operating parameters (power, mode, temperature, fan speed, swing, presets), and the AC unit responds with status frames containing the full device state including temperatures, compressor data, humidity readings, and diagnostic flags.

This specification may not cover every possible command or status field. Fields marked as "reserved" have unknown function and should be left at their default values.

# 2. Physical Layer

| Parameter | Value |
|---|---|
| Bus Type | Serial UART |
| Baud Rate | 9600 |
| Data Bits | 8 |
| Parity | None |
| Stop Bits | 1 (implied) |
| AC Connector | W950 (WiFi module port) |
| TX Pin (ESP32-S3) | GPIO 16 (double-inverted: software + 2N2222 — net result is non-inverted) |
| RX Pin (ESP32-S3) | GPIO 17 (no software inversion — hardware-inverted by level shifter) |
| Level Shifter | 2x 2N2222 NPN transistors + resistors |

## W950 Connector

The W950 connector on the AC indoor unit's main control board (PCB05-517-V02) is the WiFi module port. A cable exits the board from this connector to the WiFi module bay. This is the interface used for serial communication with the AC unit.

## Level Shifter Circuit

The W950 connector TX and RX signals operate at 5V logic, while the ESP32-S3 GPIO pins operate at 3.3V. A simple level-shifting circuit using two 2N2222 NPN transistors and pull-up resistors is used to bridge the 5V and 3.3V voltage domains.

**Signal inversion handling:**

TX (Controller → AC): The signal is inverted twice — once in software by the UART configuration and once in hardware by the 2N2222 transistor on the TX path. The two inversions cancel out, so the AC receives a non-inverted (standard polarity) signal.

RX (AC → Controller): The AC transmits its data inverted relative to standard UART logic. The 2N2222 transistor on the RX path inverts the signal back to normal, so no software inversion is needed on the controller's RX pin.

# 3. Frame Structure

## 3.1 Start and End Delimiters

All frames are delimited by fixed byte sequences:

| Delimiter | Bytes | Description |
|---|---|---|
| Frame Start | F4 F5 | Marks the beginning of a new frame |
| Frame End | F4 FB | Marks the end of a complete frame |

## 3.2 Byte Stuffing

Since F4 is used as a framing byte, any occurrence of F4 within the payload data is "stuffed" by doubling it to F4 F4. The receiver must detect consecutive F4 F4 bytes and collapse them back to a single F4. This prevents payload data from being misinterpreted as frame delimiters.

Example: a payload byte of value 0xF4 is transmitted as F4 F4 on the wire. The sequence F4 F5 always means "frame start" and F4 FB always means "frame end".

## 3.3 Checksum Calculation

The checksum is a simple 16-bit unsigned sum of the entire payload — that is, everything between the frame start delimiter (F4 F5) and the checksum itself. In other words, the checksum covers all bytes except: the 2-byte frame start (F4 F5), the 2-byte checksum, and the 2-byte frame end (F4 FB).

Since the frame is zero-indexed, the payload starts at index 2 (right after F4 F5) and ends at index (message_size - 5). The index is message_size - 5 because we exclude the last 4 bytes (2 bytes of checksum + 2 bytes of frame end F4 FB), and the index is zero-based.

**Checksum formula:**

```
checksum = SUM( frame[2] ... frame[msg_size - 5] )

Where:
  frame[0..1]               = F4 F5 (frame start, excluded)
  frame[2..msg_size-5]      = payload (checksummed)
  frame[msg_size-4..msg_size-3] = checksum (2 bytes, big-endian)
  frame[msg_size-2..msg_size-1] = F4 FB (frame end, excluded)
```

Where msg_size = frame[4] + 9 (the length byte at index 4 plus the 9-byte overhead of start + checksum + end).

**Verification example (Power ON command):**

```
Frame: F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00
       00 00 0C 00 00 00 00 04 00 00 00 00 00 00 00 00
       00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 DF F4 FB

Sum of bytes[2..45] = 0x00+0x40+0x29+...+0x04+...+0x00 = 0x01DF
Checksum bytes: 01 DF  (verified correct)
```

## 3.4 Command Frame Layout (Controller to AC)

Command frames are sent from the ESP32 controller to the AC unit. Standard commands are 50 bytes long (length byte = 0x29). The status request is shorter at 21 bytes (length byte = 0x0C).

| Byte Index | Value | Description |
|---|---|---|
| 0-1 | F4 F5 | Frame start delimiter |
| 2 | 00 | Direction: Controller to AC |
| 3 | 40 | Fixed identifier |
| 4 | 29 or 0C | Payload length (0x29=41 for commands, 0x0C=12 for status request) |
| 5-6 | 00 00 | Reserved |
| 7-8 | 01 01 | Source address (controller) |
| 9-10 | FE 01 | Destination address (AC unit) |
| 11-12 | 00 00 | Reserved |
| 13 | 65 or 66 | Message type: 0x65=command, 0x66=status request |
| 14-15 | 00 00 | Reserved |
| 16-45 | (varies) | Command payload (30 bytes) — see Section 7 |
| 46-47 | (varies) | Checksum (16-bit, big-endian) |
| 48-49 | F4 FB | Frame end delimiter |

## 3.5 Response Frame Layout (AC to Controller)

The AC unit responds with a status frame. The expected response has a length byte of 0x8D (141), giving a total frame size of 150 bytes.

| Byte Index | Value | Description |
|---|---|---|
| 0-1 | F4 F5 | Frame start delimiter |
| 2 | 01 | Direction: AC to Controller |
| 3 | 40 | Fixed identifier |
| 4 | 8D | Payload length (141 bytes) |
| 5 | 01 | Fixed |
| 6 | 00 | Fixed |
| 7-8 | FE 01 | Source address (AC unit) |
| 9-10 | 01 01 | Destination address (controller) |
| 11 | 01 | Fixed |
| 12 | 00 | Fixed |
| 13 | 66 | Message type: status response |
| 14 | 00 | Fixed |
| 15 | 01 | Fixed |
| 16+ | (varies) | Device status data — see Section 5 |
| N-3 to N-2 | (varies) | Checksum (16-bit, big-endian) |
| N-1 to N | F4 FB | Frame end delimiter |

## 3.6 Master/Slave Communication Model

The protocol is strictly master/slave. The controller (master) initiates all communication. The AC unit (slave) never transmits spontaneously — it only sends a status response frame when it has received a query (either a status request or a command frame) from the controller. If the controller sends no queries, the AC sends nothing.

## 3.7 Command-Response Behavior

Every frame sent to the AC unit — whether a command or a status request — triggers a response. The response is always a full status frame (150 bytes, as described in Section 3.5 and Section 5). This means that after any command (power on, mode change, temperature set, etc.), the AC replies with its complete current state, allowing the controller to confirm the command was accepted and to synchronize its internal state.

There is no separate "ACK" frame. The status response is the acknowledgment. If no response is received within 3 seconds of sending a command, the transmission is considered failed. The controller should discard the pending command and return to an idle state ready for the next transmission.

## 3.8 Response Frame Transmission Timing

The AC does not send the entire 150-byte status response in one continuous burst. The transmission is split into two phases:

| Phase | Content | Timing |
|---|---|---|
| 1 — Header + Payload | Frame header and payload body (~145 bytes) | Sent ~150ms after receiving the query |
| 2 — Checksum + End | Final 5 bytes: 2-byte checksum + F4 FB end delimiter | Sent up to ~10 seconds after Phase 1 |

The delay between Phase 1 and Phase 2 corresponds to the AC's internal sensor sampling cycle. The unit begins transmitting the frame immediately with the data it has, then holds back the final bytes until it has completed a fresh sensor reading. The frame is not valid until the checksum and end delimiter arrive. This split-transmission behavior applies to all status response frames, whether triggered by a status request or by a command frame.

Important: The controller's receive timeout must account for this delay. A timeout shorter than ~10 seconds will cause valid responses to be discarded as timeouts.

## 3.9 Query Queuing During Active Response

If the controller sends a query while the AC is mid-response (i.e., during the ~10-second sensor sampling wait before the checksum and end delimiter are sent), the AC buffers the incoming query. Once the current response frame is completed, the buffered query is answered immediately with no additional sampling delay, since the sensor measurement was already performed during the previous response cycle.

This means two complete status frames can arrive in quick succession (~156ms apart at 9600 baud) if a second query was sent during the previous response window. Empirically, at least one queued query is handled this way. The controller should be prepared to receive and parse back-to-back status frames.

# 4. Command Reference

All commands share the same 16-byte header (F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00) and 4-byte footer (checksum + F4 FB). The 30-byte payload between them determines the action. Below, only the variable payload bytes are described. Byte positions refer to the full frame.

## 4.1 Power Control

| Command | Key Payload Bytes | Notes |
|---|---|---|
| Power ON | Byte 18: 0x0C, Byte 23: 0x04 | Sets run_status bits to active. Must be sent before mode commands when turning on from OFF. |
| Power OFF | Byte 17: 0x01, Byte 18: 0x04, Byte 23: 0x04, Byte 24: 0x01, Byte 29: 0x01 | Additional bytes set residual state flags. |

## 4.2 Operating Modes

Mode commands set the operating mode. Byte 18 encodes the mode as a combined bitfield. When switching modes while the unit is OFF, a Power ON command must be sent first.

| Mode | Byte 16 | Byte 17 | Byte 18 | Byte 19 (Temp) | Notes |
|---|---|---|---|---|---|
| Cool | 0x01 | 0x01 | 0x50 | 0x35 (26.5°C) | Default temp included in mode frame |
| Heat | 0x01 | 0x01 | 0x30 | 0x2F (23.5°C) | Default temp included in mode frame |
| Fan Only | 0x07 | 0x01 | 0x10 | 0x33 (25.5°C) | Fan speed in byte 16 |
| Dry | 0x01 | 0x01 | 0x70 | 0x33 (25.5°C) | Dehumidification mode |

Note: Mode commands include additional flags at bytes 31 (0x10), 32 (0x01), 33 (0x04), and 35 (0x10). These appear to be "change confirmation" flags and should be included as shown in the raw frames.

## 4.3 Fan Speed

Fan speed commands modify byte 16 (speed code) and byte 17 (0x01 = active). The speed encoding in commands differs from the status response encoding.

| Fan Speed | Command Byte 16 | Status wind_status | Notes |
|---|---|---|---|
| Auto | 0x01 | 0 (0x00) | Automatic speed control |
| Quiet / Mute | 0x03 | 2 (0x02) | Lowest noise level |
| Low | 0x05 | 10 (0x0A) | |
| Medium | 0x07 | 14 (0x0E) | |
| High / Max | 0x09 | 18 (0x12) | Maximum airflow |

## 4.4 Temperature Setting

Temperature is set via byte 19 of the command frame. The encoding formula is:

```
byte_19 = (2 * temperature_value) + 1
```

This formula applies to both Celsius and Fahrenheit values. The temperature unit must match the current display unit of the AC (see Section 4.9).

| Unit | Range | Byte 19 Range | Examples |
|---|---|---|---|
| Celsius | 16°C – 32°C | 0x21 – 0x41 | 16°C=0x21, 22°C=0x2D, 32°C=0x41 |
| Fahrenheit | 61°F – 90°F | 0x7B – 0xB5 | 61°F=0x7B, 72°F=0x91, 86°F=0xAD |

Temperature-only commands have minimal payloads: only byte 19 (temperature) and byte 23 (0x04) are non-zero. All other payload bytes are 0x00. Note that some temperature values (e.g., 16°C = 0xF4) trigger byte stuffing, making those frames 51 bytes instead of 50.

## 4.5 Swing / Vane Control

Swing control is encoded in payload byte P16 (frame byte 32), which contains independent bit fields for both vertical and horizontal swing:

| Bits | Function | Values |
|---|---|---|
| 7–6 | Vertical swing | 0xC0 (bits 11) = ON (oscillating), 0x40 (bits 01) = OFF (fixed position) |
| 5–4 | Horizontal swing | 0x30 (bits 11) = ON (oscillating), 0x10 (bits 01) = OFF (fixed position) |
| 3–0 | Unused | Set to 0 |

In each 2-bit pair, the lower bit (bit 6 for vertical, bit 4 for horizontal) acts as a "field present" flag and is always set to 1. The upper bit controls the swing state: 1 = on, 0 = off.

Both fields can be encoded simultaneously in a single byte, meaning a single command frame can set both vertical and horizontal swing state at once. However, the existing captured frames set them separately with additional supporting bytes:

| Command | Byte 31 | Byte 32 (P16) | Byte 37 | Description |
|---|---|---|---|---|
| Vertical Swing ON | 0x01 | 0xC0 | — | Enables up/down oscillation |
| Vertical Swing OFF | 0x01 | 0x40 | — | Stops at current vertical position |
| Horizontal Swing ON | — | 0x30 | 0x14 | Enables left/right oscillation |
| Horizontal Swing OFF | — | 0x10 | 0x14 | Stops at current horizontal position |

**Swing mode combinations:**

To set a combined swing mode, either send two separate commands (one for vertical, one for horizontal) or encode both bit fields in a single P16 byte. For example, P16 = 0xF0 would

encode both vertical ON (0xC0) and horizontal ON (0x30) in one frame. The separate-command approach has been tested and confirmed working; the combined single-byte approach follows logically from the bit field layout but may need validation.

## 4.6 Presets (Turbo / Energy Save)

| Preset | Byte 33 | Byte 35 | Notes |
|---|---|---|---|
| Turbo ON | 0x5C | 0x10 | Boost mode — maximum performance |
| Turbo OFF | 0x54 | 0x10 | Returns to normal operation |
| Energy Save ON | 0x34 | 0x00 | Eco mode — reduced power consumption |
| Energy Save OFF | 0x14 | 0x00 | Returns to normal operation |

To return to PRESET_NONE, send both Turbo OFF and Energy Save OFF commands.

## 4.7 Display Control

| Command | Byte 36 | Description |
|---|---|---|
| Display ON | 0xC0 | Turns on the LED display panel |
| Display OFF | 0x40 | Turns off the LED display panel |

## 4.8 Sleep Modes

| Command | Byte 16 | Byte 17 | Notes |
|---|---|---|---|
| Sleep Mode 1 | 0x05 | 0x03 | Light sleep |
| Sleep Mode 2 | 0x05 | 0x05 | Medium sleep |
| Sleep Mode 3 | 0x05 | 0x07 | Deep sleep |
| Sleep Mode 4 | 0x05 | 0x09 | Deepest sleep |
| Sleep OFF | 0x01 | 0x01 | Disables sleep mode |

## 4.9 Temperature Unit Switching

The AC can be switched between Celsius and Fahrenheit display. This changes how temperature values are interpreted in both commands and status responses.

| Command | Byte 23 | Description |
|---|---|---|
| Switch to Fahrenheit | 0x07 | Display and commands use °F |
| Switch to Celsius | 0x05 | Display and commands use °C |

## 4.10 Status Request

The status request is a short 21-byte frame that asks the AC to report its full state:

```
F4 F5 00 40 0C 00 00 01 01 FE 01 00 00 66 00 00 00 01 B3 F4 FB
```

The AC responds with a 150-byte status frame (see Section 5). Note the message type byte is 0x66 (not 0x65 used for commands). The controller polls this every 5 seconds by default.

# 5. Status Response Parsing

## 5.1 Device_Status Structure

The response payload (after the 16-byte header) maps to a packed C structure. Byte offsets below are relative to the start of the payload (byte 16 of the frame). The structure uses packed alignment (#pragma pack(1)) with no padding.

| Offset | Field | Type | Description |
|---|---|---|---|
| 0 | wind_status | uint8 | Current fan speed (see 5.3) |
| 1 | sleep_status | uint8 | Sleep mode status |
| 2 | direction/run/mode | bitfield | direction_status:2, run_status:2, mode_status:4 |
| 3 | indoor_temperature_setting | uint8 | Target temp (see 5.4) |
| 4 | indoor_temperature_status | uint8 | Current indoor temp (see 5.4) |
| 5 | indoor_pipe_temperature | uint8 | Indoor coil pipe temp |
| 6 | indoor_humidity_setting | int8 | Humidity setpoint (%). 0x80 (-128) = sensor not present. |
| 7 | indoor_humidity_status | int8 | Current humidity (%). 0x80 (-128) = sensor not present. |
| 8 | somatosensory_temperature | uint8 | Sensible / feels-like temp |
| 9 | somatosensory_compensation | bitfield | ctrl:3, compensation:5 |
| 10 | fahrenheit_comp | bitfield | fahrenheit:3, compensation:5 (see 5.8) |
| 11 | timer | uint8 | Timer setting |
| 12 | hour | uint8 | Current hour |
| 13 | minute | uint8 | Current minute |
| 14 | poweron_hour | uint8 | Scheduled ON hour |
| 15 | poweron_minute | uint8 | Scheduled ON minute |
| 16 | poweroff_hour | uint8 | Scheduled OFF hour |
| 17 | poweroff_minute | uint8 | Scheduled OFF minute |
| 18 | wind_door / drying | bitfield | wind_door:4, drying:4 |
| 19 | feature flags | bitfield | See Feature Flags table below |
| 20 | feature flags 2 | bitfield | See Feature Flags table below |
| 21 | LED / filter flags | bitfield | See Feature Flags table below |
| 22 | system flags | bitfield | eeprom, sample, time_lapse, auto_check |
| 23 | error flags | bitfield | Indoor sensor / communication errors |
| 24 | communication flags | bitfield | Peripheral communication status |
| 25 | compressor_frequency | uint8 | Current compressor Hz |
| 26 | compressor_frequency_setting | uint8 | Target compressor Hz |
| 27 | compressor_frequency_send | uint8 | Sent compressor Hz command |

| | | | |
|---|---|---|---|
| 28 | outdoor_temperature | int8 | Outdoor ambient temp (°C) |
| 29 | outdoor_condenser_temperature | int8 | Condenser coil temp (°C) |
| 30 | compressor_exhaust_temperature | int8 | Compressor exhaust (°C) |
| 31 | target_exhaust_temperature | int8 | Target exhaust temp (°C) |
| 32 | expand_threshold | uint8 | Expansion valve threshold |
| 33-44 | voltage / current data | various | UAB, UBC, UCA, IAB, IBC, ICA, bus voltage, IUV |
| 45 | outdoor flags | bitfield | wind_machine:3, outdoor_machine:1, four_way:1 |

## 5.2 Mode Status Values

| mode_status | Mode | Action (compressor running) | Action (compressor idle) |
|---|---|---|---|
| 0 | Fan Only | Fan running | Fan running |
| 1 | Heat | Heating | Idle |
| 2 | Cool | Cooling | Idle |
| 3 | Dry | Dehumidifying | Idle |

When run_status == 0, the unit is OFF regardless of mode_status.

## 5.3 Fan Speed Status Values

| wind_status | Fan Speed | Description |
|---|---|---|
| 0 | Auto | Automatic speed control |
| 2 | Quiet | Lowest noise level |
| 10 | Low | Low speed |
| 14 | Medium | Medium speed |
| 18 | High | Maximum airflow |

## 5.4 Temperature Decoding

**Indoor temperature encoding (display-mode dependent):**

The encoding of indoor-side temperature bytes (indoor_temperature_setting, indoor_temperature_status, and indoor_pipe_temperature at payload offsets 3, 4, and 5) depends on the unit's display configuration:

| Display Mode | Encoding | Example |
|---|---|---|
| Celsius | Direct integer °C — no conversion needed | Raw byte 25 = 25°C |
| Fahrenheit | Likely direct integer °F (unconfirmed) | Raw byte 77 = 77°F (?) |

When the unit is configured to display in Celsius, the temperature bytes are direct Celsius integer values with no conversion required. It is believed that when configured in Fahrenheit

mode, the AC reports raw values in Fahrenheit degrees instead, but this has not been confirmed empirically. The fahrenheit_comp byte at status payload offset 10 (see Section 5.8) indicates which encoding is in use and should be consulted to select the correct interpretation at runtime.

**Outdoor temperature encoding (always Celsius):**

The outdoor-side temperatures (outdoor_temperature, outdoor_condenser_temperature, compressor_exhaust_temperature, and target_exhaust_temperature at payload offsets 28–31) are signed integers (int8) representing direct Celsius values regardless of the display configuration. These never require unit conversion.

**Sanity bounds:**

The controller should apply sanity bounds: target temperature is accepted if 7°C < value < 33°C, and current temperature if 1°C < value < 49°C. Values outside these ranges should be discarded to prevent glitched readings from corrupting the state.

Note: The command temperature encoding (byte = 2×T + 1, described in Section 4.4) is unrelated to the status response encoding and remains the same regardless of display mode.

## 5.5 Swing Status

| left_right | up_down | Swing Mode |
|---|---|---|
| 0 | 0 | SWING_OFF |
| 0 | 1 | SWING_VERTICAL |
| 1 | 0 | SWING_HORIZONTAL |
| 1 | 1 | SWING_BOTH |

## 5.6 Sensor Data Fields

The following sensor values are available from the status response:

| Sensor | Struct Field | Unit | Notes |
|---|---|---|---|
| Compressor Frequency | compressor_frequency | Hz | Current. >0 means compressor running. |
| Compressor Freq. Setting | compressor_frequency_setting | Hz | Target frequency |
| Compressor Freq. Send | compressor_frequency_send | Hz | Last sent command |
| Outdoor Temperature | outdoor_temperature | °C | Ambient outdoor (signed int8, always °C) |
| Condenser Temperature | outdoor_condenser_temperature | °C | Outdoor coil (signed int8, always °C) |
| Exhaust Temperature | compressor_exhaust_temperature | °C | Compressor exhaust (signed int8, always °C) |
| Target Exhaust Temp | target_exhaust_temperature | °C | Target for exhaust regulation (always °C) |
| Indoor Pipe Temperature | indoor_pipe_temperature | °C | Indoor coil pipe temp (see 5.4 for encoding) |
| Humidity Setpoint | indoor_humidity_setting | % | signed int8. 0x80 (-128) = not present. |
| Humidity Status | indoor_humidity_status | % | signed int8. 0x80 (-128) = not present. |

**Humidity sensor sentinel value:**

The indoor_humidity_setting and indoor_humidity_status fields return the value 0x80 (-128 decimal) when the unit has no humidity sensor installed or humidity data is unavailable. This sentinel value should be treated as "not present" and must not be interpreted as a real humidity reading.

## 5.7 Feature Flags (Offset 19, Byte 20)

| Bit | Field | Description |
|---|---|---|
| 7 | up_down | Vertical swing active |
| 6 | left_right | Horizontal swing active |
| 5 | nature | Natural wind mode |
| 4 | heat | Heating air active |
| 3 | low_power | Energy saving mode |
| 2 | low_electricity | Save electricity mode |
| 1 | efficient | Efficient mode |
| 0 | dual_frequency | Dual frequency mode |

## 5.8 Fahrenheit/Compensation Byte (fahrenheit_comp)

The byte at status payload offset 10 is a bitfield with two sub-fields: fahrenheit (bits 7–5, 3 bits) and compensation (bits 4–0, 5 bits). This byte indicates the temperature display mode of the AC unit and affects how indoor temperature bytes should be interpreted (see Section 5.4).

| Display Mode | Full Byte Value | fahrenheit Field (bits 7–5) | Notes |
|---|---|---|---|
| Celsius | 0x81 (binary 10000001) | 4 (0b100) | Confirmed empirically |
| Fahrenheit | Unknown | Unknown | Not yet confirmed |

When the AC display is configured in Celsius mode, this byte carries the value 0x81. The top 3-bit fahrenheit field equals 4 (0b100) in this case. The value this field carries in Fahrenheit display mode has not yet been confirmed empirically and remains unknown. The controller should read this field to determine whether indoor temperature bytes require unit conversion.

# 6. State Machine and Timing

The controller uses a simple state machine for UART communication with two states:

| State | Description | Transition |
|---|---|---|
| IDLE | No pending transmission. Ready to send next queued command. | Transitions to WAITING_ACK when a command is sent. |
| WAITING_ACK | A command has been sent, waiting for the AC to respond. | Transitions to IDLE when F4 F5 response start is detected, or after 3-second timeout. |

**Timing constraints:**

Minimum inter-message gap: 100ms between consecutive sends. Minimum gap after last received byte: 10ms. Post-send idle period: 1500ms after each transmission before the next is allowed. ACK timeout: 3000ms — if no response header (F4 F5) is received within 3 seconds, the send state resets to IDLE, the bus is returned to idle mode, and the failed message is dequeued. Status polling interval: 5000ms (recommended default, configurable).

**Command queueing:**

Commands are placed in a FIFO queue and sent one at a time. When multiple commands are needed (e.g., turning on from OFF requires Power ON + Mode Set + Temperature Set), they are queued sequentially and the timing constraints ensure proper spacing between them.

**Startup behavior:**

On startup, the UART receive buffer is flushed to clear any stale data. The first action is a status request to synchronize the controller state with the AC unit. After receiving a response, the remaining RX buffer is flushed again to discard any stale bytes from the UART bus.

# 7. Command Payload Byte Map

This section maps each byte position in the 30-byte command payload (frame bytes 16–45) to its function. Bytes not listed are always 0x00 for that command type.

| Payload Byte | Frame Byte | Function | Used By |
|---|---|---|---|
| P0 | 16 | Fan speed / wind code | Fan speed, Sleep, Mode |
| P1 | 17 | Sleep code / active flag | Sleep, Mode, Fan, Preset |
| P2 | 18 | Mode + Run bitfield | Power, Mode |
| P3 | 19 | Temperature value (2*T+1) | Temperature, Mode (default temp) |
| P7 | 23 | Control class (0x04=normal, 0x05=°C, 0x07=°F) | Most commands, Unit switch |
| P8 | 24 | Off-state flag | Power OFF |
| P13 | 29 | Off-state residual | Power OFF |
| P15 | 31 | Vertical vane base | Vertical swing |
| P16 | 32 | Swing mode flags | Vertical swing, Horizontal swing |
| P17 | 33 | Preset / turbo flags | Turbo, Energy Save |
| P19 | 35 | Preset confirm / fan confirm | Turbo, Fan speed, Mode |
| P20 | 36 | Display control | Display ON/OFF |
| P21 | 37 | Horizontal vane value | Horizontal swing |

# 8. Appendix: Full Frame Listing

Complete hex dumps of all known command frames, listed by function.

### Power ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 0C 00 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 DF F4 FB
```

### Power OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 01 04 00 00 00 00 04 01 00 00 00 00
01 00 00 01 55 00 01 00 00 00 00 00 00 00 00 00 00 02 31 F4 FB
```

### Mode: Cool

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 50 35 00 00 00 04 00 00 00 00 00
00 00 10 01 04 00 10 00 00 00 00 00 00 00 00 00 00 02 7F F4 FB
```

### Mode: Heat

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 30 2F 00 00 00 04 00 00 00 00 00
00 00 10 01 04 00 10 00 00 00 00 00 00 00 00 00 00 02 59 F4 FB
```

### Mode: Fan Only

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 07 01 10 33 00 00 00 04 00 00 00 00 00
00 00 10 01 04 00 10 00 00 00 00 00 00 00 00 00 00 02 43 F4 FB
```

### Mode: Dry

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 70 33 00 00 00 04 00 00 00 00 00
00 00 10 01 04 00 10 00 00 00 00 00 00 00 00 00 00 02 9D F4 FB
```

### Fan: Auto

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 04 00 10 00 00 00 00 00 00 00 00 00 00 01 E9 F4 FB
```

### Fan: Quiet

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 03 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 04 00 30 00 00 00 00 00 00 00 00 00 00 02 0B F4 FB
```

### Fan: Low

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 05 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 04 00 10 00 00 00 00 00 00 00 00 00 00 01 ED F4 FB
```

### Fan: Medium

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 07 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 04 00 10 00 00 00 00 00 00 00 00 00 00 01 EF F4 FB
```

### Fan: High

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 09 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 04 00 10 00 00 00 00 00 00 00 00 00 00 01 F1 F4 FB
```

### Turbo ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 5C 00 10 00 00 00 00 00 00 00 00 00 00 02 41 F4 FB
```

### Turbo OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 01 01 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 54 00 10 00 00 00 00 00 00 00 00 00 00 02 39 F4 FB
```

### Energy Save ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 34 00 00 00 00 00 00 00 00 00 00 00 00 02 07 F4 FB
```

### Energy Save OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 14 00 00 00 00 00 00 00 00 00 00 00 00 01 E7 F4 FB
```

### Display ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 C0 00 00 00 00 00 00 00 00 00 00 02 93 F4 FB
```

### Display OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 00 00 00 02 13 F4 FB
```

### Vertical Swing ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 01 C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 94 F4 FB
```

### Vertical Swing OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 01 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 14 F4 FB
```

### Horizontal Swing ON

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 30 00 00 00 00 14 00 00 00 00 00 00 00 00 00 02 17 F4 FB
```

### Horizontal Swing OFF

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 04 00 00 00 00 00
00 00 00 10 00 00 00 00 14 00 00 00 00 00 00 00 00 00 01 F7 F4 FB
```

### Switch to Fahrenheit

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 07 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 D6 F4 FB
```

### Switch to Celsius

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 00 00 00 00 05 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 D4 F4 FB
```

### Status Request

```
F4 F5 00 40 0C 00 00 01 01 FE 01 00 00 66 00 00 00 01 B3 F4 FB
```

### Temperature Commands (selected examples)

*Temp 16°C (byte 19=0x21)*

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 21 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 F4 F4 F4 FB
```

*Temp 22°C (byte 19=0x2D)*

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 2D 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 F4 FB
```

*Temp 32°C (byte 19=0x41)*

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 41 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 14 F4 FB
```

*Temp 72°F (byte 19=0x91)*

```
F4 F5 00 40 29 00 00 01 01 FE 01 00 00 65 00 00 00 00 00 91 00 00 00 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 64 F4 FB
```

# Disclaimer

This document is an unofficial, community-derived protocol specification. It is not produced, endorsed, authorized, or affiliated with Panavox, Hisense, Aircon International, or any of their subsidiaries or affiliates. The Panavox and Hisense names and trademarks are the property of their respective owners and are used here solely for identification purposes.

**Origin of this specification:**

This specification was derived from the open-source ESPHome custom component for Hisense/Aircon International compatible air conditioning units, available at:

```
https://github.com/pslawinski/esphome_airconintl
```

The protocol knowledge in this document originates from that codebase. Additional corrections and findings were obtained through empirical testing on a physical Panavox PSS-12 INV unit connected via the W950 WiFi module connector.

**Accuracy and completeness:**

This specification is based on reverse engineering and may contain errors, omissions, or inaccuracies. Some byte positions and values may have additional undocumented functions. Not all features of the protocol have been confirmed empirically, and behavior may vary across different hardware revisions, firmware versions, or regional variants of the AC unit. Fields and behaviors explicitly marked as "unconfirmed" or "unknown" have not been validated on real hardware.

**Liability:**

This document is provided "as is" without warranty of any kind, express or implied. The authors accept no liability for any damage, malfunction, or voided warranty resulting from the use of this specification. Connecting third-party hardware to the AC unit’s internal control board may void the manufacturer’s warranty and carries inherent risks including but not limited to damage to the AC unit, the controller hardware, or connected systems. Use at your own risk.

**License:**

The upstream source code from which this specification is derived is subject to its own license terms. Please consult the repository for details.

# Document History

| Version | Date | Author | Changes |
|---|---|---|---|
| 1.0 | 2026-03-31 | Farid Elias | Initial protocol specification for W950 serial interface |